

Sovereign On-Premise Agentic AI Workbench — Final Implementation PRD (v2)

SIH 2026 · Problem Statement 26117 · Organization: MRPL (Mangalore Refinery and Petrochemicals Limited) · Theme: Smart Automation Team shape: 2 Backend Engineers + 1 ML Engineer Supersedes: [ps26117-technical-requirements-and-overview.pdf](#) , [SIH26117_Enhanced_Research_Document.pdf](#) , and v1 of this Implementation PRD. Verified/updated: 23 August 2026.

What changed from v1 (read this first)

#	Change	Reason
1	Confirmed, unchanged: single-workstation/server deployment, no Kubernetes/HA, no multi-node.	PS text says explicitly: <i>"demonstrable on a single workstation or server."</i> v1 already had this right.
2	Scope addition: a real structured P&ID connectivity/CAD-graph parser is now in scope (Section 6, FR-4B, Workflow 4). Previously this was excluded in both v1 and the research synthesis's "what not to build" list.	This exclusion existed only because the <i>research documents</i> recommended cutting it for hackathon-speed reasons — the PS itself never excludes it, and P&IDs are the PS's own headline example of confidential data (<i>"Piping & Instrument Diagrams... review of scanned drawings"</i>). Explicit decision: build it.
3	Confirmed, unchanged: every other item on the "do not build" list stays excluded — RBAC/SSO, multi-agent swarms, custom inference engine, custom vector DB, fine-tuning, browser automation, giant/frontier model, production Kubernetes.	These were excluded because the PS doesn't ask for them, not just because the research doc said so. Adding them back would burn build time the PS doesn't reward.
4	Gap closed: the PS explicitly says deliverables include <i>"calculations with steps shown"</i> and <i>"PPT/Word/Excel files."</i> v1's Workflow 1 only produced a DOCX with prose reasoning — no explicit step-by-step calculation artifact, and PPTX/XLSX were registered tools but never exercised end-to-end. Fixed in FR-7 and Workflow 1.	Closing this gap directly hits PS wording the earlier draft under-served.

Everything else in v1 (team ownership split, laptop-tier model roster, agent state machine, sandbox hardening, sovereignty proof plane, audit log, folder structure) was already sound and is carried forward unchanged unless noted.

1. PS Requirement Traceability Matrix

This table exists so nobody has to argue, mid-build, about whether something is "in scope." If it's a row here, it ships. If it's not a row here, it's in Section 13 (Non-Goals) on purpose.

PS requirement (paraphrased from the actual PS text)	Delivered by	Spec section
Self-hosted, air-gapped, nothing leaves the premises	Sovereignty Proof Plane, egress-denial at host + container level	FR-8, Workflow 3
Not locked to one model; multiple open-weight models; auto-pick the right one per task	Model Registry + Router	FR-1
New models addable later without redesigning the system	Registry-driven router + provider/adaptor interface (<code>models/adapters/</code>)	§4, FR-1
Agent plans multi-step work and iterates , not one-shot chat	Bounded deterministic state machine with REPAIR loop	§5.1, FR-2
Local tools: file read/write, code execution sandbox, spreadsheet work, internal document search	Typed Tool Registry, allow-listed per task type	FR-3, FR-6
Handle scanned PDFs, handwritten notes, photographs via on-device OCR/vision	Multimodal Ingestion Pipeline	FR-4
Handle engineering drawings / P&IDs specifically (PS's own headline example of confidential data)	P&ID Structured Connectivity Parser (new)	FR-4B, Workflow 4
Real deliverables: approval notes, PPT/Word/Excel , working code, calculations with steps shown	Artifact Generation, with explicit calc-steps table now wired into Workflow 1	FR-7, Workflow 1
Ground itself in the org's own manuals/SOPs/past correspondence, nothing external	Local RAG with mandatory citations	FR-5
Demonstrable on a single workstation/server, mid-range GPU (smaller model OK if hardware is weaker)	Laptop-tier hardware & model strategy	§4
Model auto-selection shown across at least two task types	Routing receipt, visible in every response	FR-1
Agentic task end-to-end: scanned inspection report → key findings → approval note as Word file	Workflow 1	§7
Coding task run and verified in a sandbox	Workflow 2	§7

PS requirement (paraphrased from the actual PS text)	Delivered by	Spec section
Multimodal task involving image or scanned document understanding	Workflow 1 (OCR/VLM) and Workflow 4 (P&ID graph) — the latter goes well beyond the minimum bar	§7
Prove — via logs or a visible network monitor — that no external calls are made	Sovereignty Monitor + live-blocked-request demo	FR-8/FR-9, Workflow 3

Nothing in the PS's "Expected Solution" paragraph is left unmapped.

2. Product Decision

Core principle, unchanged: build a **narrow, workflow-oriented, approval-gated, air-gapped agent runtime** — not a generic local-ChatGPT clone. Judges reward one thing done completely end-to-end far more than ten things half-wired.

What we are building — a local-only "AI Workbench" where a user in a confidential industrial setting (refinery / PSU / defence org) can:

1. Upload a scanned inspection report → get a cited, SOP-grounded, human-reviewable approval-note **DOCX** **plus a calculation-steps XLSX**.
2. Ask for a coding change → get sandboxed, tested, self-repaired, working code.
3. Upload a P&ID → get a **structured connectivity graph** (not just a caption) plus a grounded flow-path narrative.
4. See — not just be told — that the entire system never touches the internet.

Everything else non-PS (full RBAC/SSO, multi-user, multi-agent swarms, SSO, model fine-tuning, custom inference engines, production Kubernetes/HA) is explicitly out of scope. See Section 13.

3. Team Ownership

Split by subsystem, not by workflow — each workflow crosses all three people's subsystems anyway, so integration happens continuously rather than as one late merge.

Owner	Owns	Does NOT own
ML Engineer	Model registry, model router, Ollama model management, RAG pipeline (ingestion → chunking → embeddings → Qdrant → retrieval → citations), OCR/ vision pipeline, P&ID symbol detection + line tracing + graph assembly , prompt/plan templates, evaluation of model outputs	API framework, frontend, sandbox, firewall
Backend Dev A		

Owner	Owns	Does NOT own
	FastAPI gateway, agent orchestrator (state machine), tool registry + execution, artifact generators (DOCX/PPTX/XLSX), P&ID tool wrapper + graph JSON schema + API endpoint , audit log, task/session data model	Model internals, RAG internals, frontend
Backend Dev B	Frontend (Workbench UI), code sandbox executor, sovereignty/network-isolation layer, Docker Compose + deployment, telemetry dashboard, P&ID graph/overlay rendering in the UI	Model internals, RAG internals

Frozen interfaces (contracts, not conversations):

- ML → Backend A: `route_request(task) -> ModelHandle` , `retrieve(query, top_k) -> List[Chunk]` , `extract_pid_graph(image) -> PIDGraph` . Backend A never calls Ollama or the CV pipeline directly.
- Backend A → Backend B: `execute_in_sandbox(code, tests) -> ExecutionResult` . The orchestrator never calls Docker directly.
- Backend A → Frontend: the REST/streaming API in Section 8. Frontend never reads agent internals directly.

4. Hardware & Model Strategy (laptop-tier, single node)

Confirmed per PS wording ("single workstation or server," "use a smaller open-weight model if 120B-class hardware isn't available"): single-node only. No Kubernetes, no multi-node HA, no cluster orchestration — see Non-Goals. The research synthesis's 24GB-desktop-GPU assumption does not apply to this team; we design for the actual laptop GPU and treat anything better as headroom.

4.1 Design decision: one LLM resident at a time

A laptop GPU (commonly 4–8GB VRAM on RTX 40-series mobile parts, up to 12–16GB on high-end mobile 4080/4090) cannot hold a general model + coder model + vision model simultaneously.

- Only one LLM is loaded in VRAM at any moment.
- The Model Router decides which model a task needs and tells Ollama to load it (`keep_alive`), evicting the previous one if VRAM is tight.
- Small, cheap models stay resident (embeddings) since they're inexpensive; large generative models swap.

This is not a compromise to hide — it's the demo story: *"watch the router evict the reasoning model and load the vision model because a P&ID scan just came in"* is a more convincing agentic-routing demo than three idle models.

4.2 Model roster (verified current as of Aug 2026)

Role	Model (primary)	Size @ Q4_K_M	Fallback (tighter VRAM)
General reasoning / planning / drafting	<code>qwen3:8b-instruct</code>	~5 GB	<code>qwen3:4b-instruct</code> (~2.5 GB)

Role	Model (primary)	Size @ Q4_K_M	Fallback (tighter VRAM)
Coding	qwen2.5-coder:7b-instruct	~4.5 GB	qwen2.5-coder:3b-instruct
Vision / OCR fallback / P&ID cross-check	qwen2.5-vl:7b-instruct	~5 GB	qwen2.5-vl:3b-instruct (~2.5 GB)
Embeddings (resident, CPU is fine)	bge-m3	~1.2 GB	nomic-embed-text

All pulled and served through **Ollama**. Wrapped behind a thin provider interface (`models/adapters/`) so vLLM/ llama.cpp can be substituted later without touching router or agent code — never call a model runtime directly from application code.

Do **not** use Qwen3-Coder-30B-A3B or any 27B+/MoE model on this hardware tier — even "3B active" MoE models need the full parameter set resident or CPU-offloaded, and CPU offload on a laptop is slow enough to break a live demo.

4.3 P&ID pipeline is not an LLM-VRAM cost

The symbol-detection/line-tracing stage (Section 6, FR-4B) is classical CV plus, optionally, a tiny object detector (YOLOv8n, ~6MB). It runs on CPU or a sliver of GPU independent of the single-resident-LLM budget — it does not compete with the router's VRAM management. Only the final cross-check/narrative step calls the already-resident vision LLM.

4.4 Hard fallback

Keep a CPU-only path alive at all times (`qwen3:4b` and `qwen2.5-vl:3b` both run tolerably on CPU). If the demo machine's GPU driver misbehaves, the system should still answer — just slower. Test this explicitly.

4.5 Model Registry

Central JSON/YAML registry, loaded at startup, is the single source of truth for what the router can pick from:

```
{
  "models": [
    {
      "id": "general-reasoning",
      "name": "qwen3:8b-instruct",
      "type": "reasoning",
      "modalities": ["text"],
      "tasks": ["planning", "analysis", "drafting", "classification"],
      "context_length": 32768,
      "quantization": "Q4_K_M",
      "vram_gb": 5.0,
      "endpoint": "local://ollama/qwen3:8b-instruct",
      "priority": 1,
      "available": true
    },
    {
      "id": "coding",
      "name": "qwen2.5-coder:7b-instruct",
      "type": "coding",
      "modalities": ["text"],
      "tasks": ["codegen", "debugging", "testing"],
      "context_length": 32768,
      "quantization": "Q4_K_M",
      "vram_gb": 4.5,
      "endpoint": "local://ollama/qwen2.5-coder:7b-instruct",
      "priority": 1,
      "available": true
    },
    {
      "id": "vision",
      "name": "qwen2.5-vl:7b-instruct",
      "type": "vision",
      "modalities": ["text", "image"],
      "tasks": ["ocr_fallback", "image_analysis", "pid_cross_check", "pid_narrative"],
      "context_length": 16384,
      "quantization": "Q4_K_M",
      "vram_gb": 5.0,
      "endpoint": "local://ollama/qwen2.5-vl:7b-instruct",
      "priority": 1,
      "available": true
    },
    {
      "id": "embeddings",
      "name": "bge-m3",
      "type": "embedding",
      "modalities": ["text"],
      "tasks": ["embed"],
      "endpoint": "local://ollama/bge-m3",
      "priority": 1,
      "available": true
    }
  ]
}
```

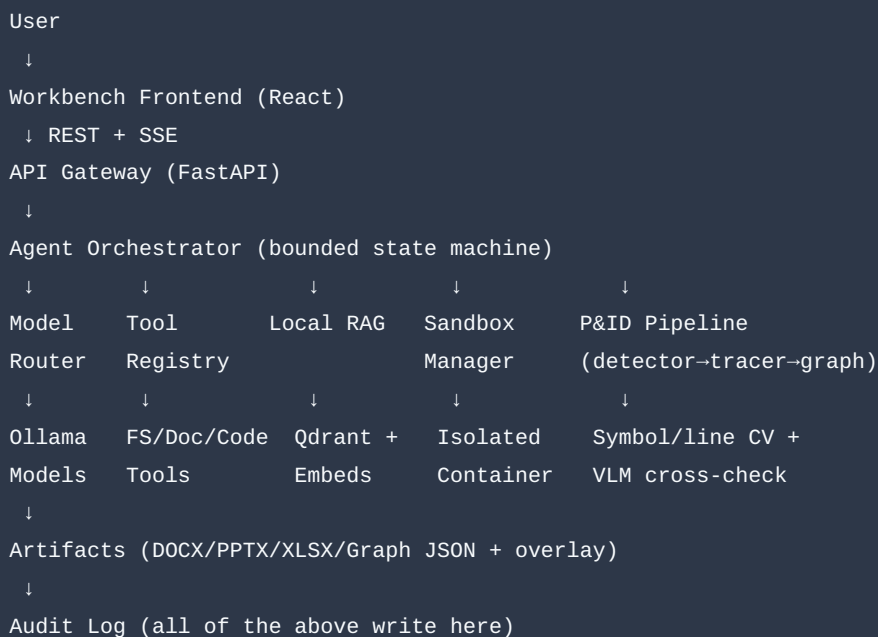
```

    }
  ]
}

```

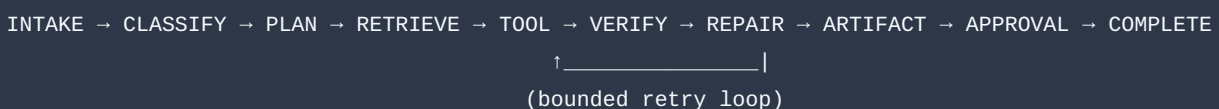
The application talks to `models/router`, never to individual model names. Adding, swapping, or resizing a model means editing this file — nothing else.

5. System Architecture



5.1 Agent state machine (deterministic, not a free-running loop)

An explicit, bounded state machine — not an unrestricted ReAct/agent-swarm loop. More reliable, more observable, and dramatically easier to demo/debug under time pressure.



Guarantees every state must uphold:

- Typed input/output per transition (pydantic models, not free-form dicts).
- A max-step counter — hard stop with a clear error state, never an infinite loop.
- A per-state timeout.
- Tools are allow-listed per task type — the coding task cannot call `create_docx`, the inspection task cannot call `execute_code`, only the P&ID task can call `extract_pid_graph`.

- Retrieved RAG content is data, never instructions — never concatenate retrieved text into a system/instruction role.
- Any destructive or "decision" action (approval note, final recommendation) stops at an **APPROVAL** state and waits for human sign-off before being marked complete. The AI recommends; a human approves. This is a scored PS requirement, not a nice-to-have.

5.2 Task state object

```
{
  "task_id": "uuid",
  "user_request": "string",
  "task_type": "inspection | coding | pid_analysis | general",
  "plan": ["step1", "step2", "..."],
  "current_state": "RETRIEVE",
  "model_used": "general-reasoning",
  "tool_calls": [
    {"tool": "ocr_document", "input": "...", "output": "...", "timestamp": "..."}
  ],
  "retrieved_chunks": [
    {"source": "SOP-ENG-042", "section": "4.2", "page": 17, "text": "..."}
  ],
  "artifacts": ["approval_note.docx", "calculation_sheet.xlsx"],
  "pid_graph": null,
  "errors": [],
  "verification_status": "pending | passed | failed",
  "final_result": null,
  "requires_human_approval": true,
  "approved_by": null
}
```

5.3 Backend folder structure

```
backend/
├─ api/                # FastAPI routers: chat, files, tasks, artifacts, monitoring, pid
├─ agent/
│   ├─ planner.py      # builds `plan` from user_request + task_type
│   ├─ executor.py     # runs the state machine
│   ├─ memory.py       # per-task_state persistence (SQLite for the prototype)
│   └─ verifier.py     # checks artifact/code/graph validity before ARTIFACT→APPROVAL
├─ models/
│   ├─ registry.py     # loads model_registry.json
│   ├─ router.py       # classify() + select() + Ollama load/unload
│   └─ adapters/       # ollama_adapter.py (swap for vllm_adapter.py later)
├─ tools/
│   ├─ filesystem.py   # read_file, write_file, list_files, search_files
│   ├─ documents.py    # extract_pdf_text, ocr_document, analyze_pdf_page
│   ├─ knowledge.py    # search_knowledge_base, retrieve_document, retrieve_section
│   ├─ code.py         # create_code, execute_code, run_tests
│   └─ vision.py      # analyze_image
├─ pid/               # NEW – structured P&ID connectivity pipeline
│   ├─ detector.py     # symbol/instrument detection (template match / tiny YOLO)
│   ├─ tracer.py       # line/pipe tracing (skeletonize + connected components)
│   ├─ graph.py        # assemble nodes/edges into PIDGraph, confidence scoring
│   └─ overlay.py      # render annotated overlay image for the UI
├─ rag/
│   ├─ ingestion.py    # PDF/image → text/layout normalization
│   ├─ embeddings.py   # bge-m3 calls
│   ├─ retrieval.py     # Qdrant hybrid search + rerank (optional)
│   └─ citations.py     # attach source/section/page to every chunk returned
├─ sandbox/
│   ├─ executor.py     # spins up a hardened, ephemeral Docker container
│   └─ limits.py       # cpu/mem/pids/timeout constants
├─ artifacts/
│   ├─ docx.py / pptx.py / xlsx.py # python-docx / python-pptx / openpyxl generators
│   └─ validators.py   # structural validation per format
└─ security/
    ├─ auth.py         # local single-tenant auth
    ├─ audit.py        # append-only JSONL audit log
    └─ network.py      # sovereignty enforcement + live metrics
```

6. Functional Requirements

FR-1 — Model Registry & Router

Load `model_registry.json`, expose `classify(request) -> task_type` and `select(task_type, modality) -> model_id`.

Router logic — hybrid, deterministic-first: 1. Rule-based fast path: file attached is an image/scanned PDF → `vision`. File is specifically flagged/detected as a P&ID → `vision` + route to `pid` pipeline. Request contains code/programming keywords or a code-extension file → `coding`. Otherwise → `reasoning`. 2. Only if the rule-based path is ambiguous, use one cheap classification call to the currently-loaded model.

Acceptance: every response shows a "routing receipt" — which model handled the request and why (rule matched / LLM classified). This single artifact demonstrates FR-1, FR-2, and part of auditability at once — build it early, and make sure the demo shows it firing across **at least two distinct task types** (PS's explicit ask).

FR-2 — Agent Orchestrator

Implements the state machine in §5.1 over the task object in §5.2. Every transition writes one line to the audit log. Bounded retry: on VERIFY failure, go to REPAIR at most N times (config constant, e.g. 3) before failing the task cleanly — never a silent hang.

FR-3 — Tool Registry

Typed functions with explicit allow-lists per task type.

Tool	Purpose	Used by workflow
<code>read_file</code> / <code>write_file</code> / <code>list_files</code> / <code>search_files</code>	Workspace-jailed filesystem access	all
<code>extract_pdf_text</code>	Text-layer PDF extraction	1
<code>ocr_document</code>	OCR for scanned/ handwritten pages	1
<code>analyze_pdf_page</code> / <code>analyze_image</code>	VLM-based reading of scans	1
<code>extract_pid_graph</code>	NEW — structured P&ID connectivity extraction	4
<code>search_knowledge_base</code> / <code>retrieve_document</code> / <code>retrieve_section</code>	RAG	1
<code>create_docx</code> / <code>create_pptx</code> / <code>create_xlsx</code>	Artifact generation	1, 3
<code>create_code</code> / <code>execute_code</code> / <code>run_tests</code>	Coding workflow	2

Every tool call is typed (pydantic input/output schema) and logged with input, output, and duration.

FR-4 — Multimodal Ingestion Pipeline

PDF/Image input

- ↓ detect text layer
- ↓ if scanned/no text layer → render pages to images
- ↓ OCR (Tesseract, primary – see §14 rationale)
- ↓ if OCR confidence low OR handwriting detected → VLM fallback (qwen2.5-v1)
- ↓ normalize to structured text + bounding boxes
- ↓ hand to Agent

Tesseract stays primary over PaddleOCR for install-reliability on unknown laptop OS/driver combos — this is unchanged from v1 and still the right call.

FR-4B — P&ID Structured Connectivity Parser (NEW, in scope per this decision)

This is the scope addition. It goes beyond "describe what you see" to producing an actual machine-readable connectivity graph — the kind of artifact an engineer could plausibly use.

P&ID image/scan

- ↓ preprocess (deskew, binarize, line enhancement)
- ↓ symbol/instrument detection
 - primary: template/contour matching against a small library of standard ISA-5.1 symbols (valves, pumps, vessels, instruments) – fast, works well on the clean vector-style P&IDs in the public sample datasets
 - stretch (only if time allows): YOLOv8n fine-tuned on an open P&ID symbol dataset for better generalization to noisier scans
- ↓ line/pipe tracing: skeletonize the drawing, trace connected line segments between symbol bounding boxes (Hough transform / connected-component analysis) to build edges
- ↓ label association: OCR (Tesseract) reads tags/instrument numbers near each symbol, binds each tag to its nearest node
- ↓ graph assembly:
 - nodes = {id, symbol_type, tag, bbox, confidence}
 - edges = {from_node, to_node, line_type, confidence}
- ↓ VLM cross-check (qwen2.5-v1): given the original drawing + the assembled graph, the model is asked to confirm/flag likely-wrong connections and generate the natural-language "explain the flow path" narrative – the model reasons over the extracted graph as grounding data, not raw pixels alone, which keeps the narrative traceable back to a specific edge/node rather than free-hallucinated
- ↓ confidence gate: below-threshold nodes/edges are still shown (greyed out) rather than silently dropped or guessed; if overall graph confidence is low, the system explicitly falls back to VLM-only description for the low-confidence regions – same fallback v1 already specified, now used as a safety net rather than the primary path
- ↓ output: structured JSON graph + annotated overlay image (nodes/edges drawn on the original scan) + grounded narrative – all shown in the Artifacts/Context panel

Honest scope note: what's in scope is a structured parser *tuned against a curated set of public P&ID fixtures* (the PS explicitly says to use "publicly available document samples... sample P&IDs from open datasets"). A general-purpose parser that reliably handles arbitrary, messy, hand-annotated field scans is not realistic in this timeline and stays out of scope (see §13). Pick 2–3 known-good public P&ID samples, tune the detector/tracer against exactly those, and rehearse on those — same "seed the fixture" philosophy already used for Workflows 1 and 2.

Acceptance: given one of the curated P&ID fixtures, the pipeline produces a graph JSON with node/edge count matching a hand-verified ground truth within an agreed tolerance, an overlay image a human can visually sanity-check in seconds, and a narrative that only references nodes/edges actually present in the graph (spot-checkable — the VLM must not invent equipment that isn't in the extracted graph).

FR-5 — Local RAG

SOP/manual documents

- ↓ parse (PyMuPDF for text PDFs; ingestion pipeline above for scans)
- ↓ chunk (semantic/section-aware, not fixed-token, so citations map to real sections)
- ↓ embed (bge-m3)
- ↓ store in Qdrant (single Docker container, one collection per corpus type)

User request

- ↓ embed query
- ↓ Qdrant similarity search (top-k, optionally hybrid dense+sparse — bge-m3 supports both)
- ↓ return chunks WITH source/section/page metadata attached
- ↓ reasoning model drafts an answer that must cite retrieved chunks

Acceptance: every generated recommendation in Workflow 1 shows a citation block (`Source: SOP-ENG-042, Section 4.2, Page 17`) — this is what makes the output traceable rather than a hallucination. Do not skip it for time.

FR-6 — Sandboxed Code Execution

Generated code never runs on the host. Hardened standard Docker (not rootless Podman — correct guarantees, far less setup overhead for a 3-person team):

- `--network=none` (no network, period)
- `--cap-drop=ALL --security-opt=no-new-privileges`
- `--memory=512m --cpus=1 --pids-limit=64`
- `--read-only` root filesystem, with only a `/workspace` tmpfs mount writable
- hard wall-clock execution timeout, container killed and removed after every run (ephemeral, never reused)
- default seccomp profile (don't hand-roll a custom profile unless there's spare capacity)

Flow: generate → execute → run tests → tests fail → repair (bounded retries) → tests pass → verified result.

FR-7 — Artifact Generation

`python-docx`, `python-pptx`, `openpyxl` for DOCX/PPTX/XLSX generation from templates. Every artifact passes through a validator before being marked complete (file opens without error, required sections/fields present).

Closing the v1 gap — calculations with steps shown (PS wording, verbatim): Workflow 1's output is no longer DOCX-only. It now emits: - `approval_note.docx` — Recommendation + Source Citation + Reviewer field (as before). - `calculation_sheet.xlsx` — a real spreadsheet showing the deterministic calculation used to check the measured value against the SOP tolerance, one row per step, with the final pass/fail flag and a link back to the SOP citation. This is a *deterministic* calculation (plain arithmetic against the retrieved tolerance value), not an LLM-narrated one — the LLM explains it, but does not compute it, so the number is trustworthy.

This also means the demo genuinely exercises **DOCX and XLSX** end-to-end (the PS's own "PPT/Word/Excel" phrasing), not just DOCX with the others left as unexercised registered tools. PPTX (`create_pptx`) stays available as an on-demand tool (e.g. "summarize this as a one-slide brief") but is not required to be part of a scored end-to-end workflow.

Artifacts are shown in a dedicated Artifacts panel — separate from the chat transcript.

FR-8 — Sovereignty Proof Plane

Scored, load-bearing requirement, not a UI skin. "100% Local" must be enforced, not displayed.

- **Container level (primary, easiest to get right):** sandbox always runs `--network=none` . Cheap to demo live — attempt an outbound `curl` inside the sandbox on stage, show it fail instantly.
- **Host level:** block outbound traffic for the whole app process during the demo.
- Linux (or Docker Desktop on Windows via WSL2, a real Linux kernel): `nftables` / `iptables` default-deny egress on the app's network namespace, or run the whole stack inside a Compose network with no external gateway.
- If a genuinely offline demo isn't feasible on the demo machine's OS: OS firewall rules blocking the app's binary (Windows Defender Firewall outbound rule) — weaker but still real enforcement.
- **Telemetry:** `tcpdump` (or a lightweight Python socket-level counter if `tcpdump` isn't installable) counts and logs any attempted outbound connection, DNS query, blocked packet. Feeds a live dashboard:

```
SOVEREIGNTY MONITOR
Internet Access: BLOCKED
External API Calls: 0
External DNS Queries: 0
External Connections: 0
Local Requests: <live counter>
Status: AIR-GAPPED
```

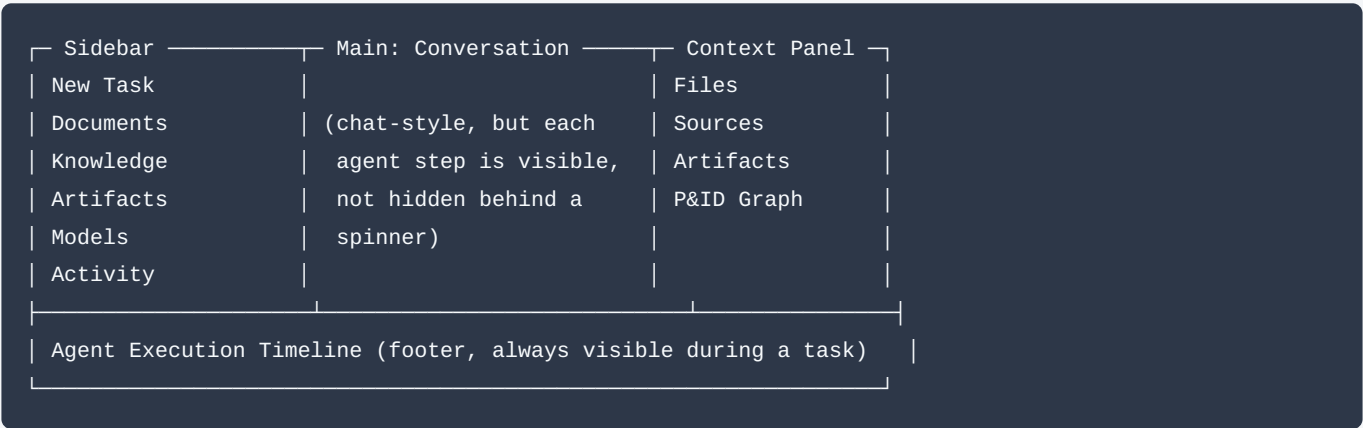
Best demo beat: deliberately trigger an outbound request from inside the sandbox live, show it blocked and logged in real time, then continue the normal workflow with zero external traffic.

FR-9 — Security & Auditability

- Auth: single local account is enough for this scope. RBAC/SSO stays out of scope, but a `role` field already exists on the user/session object so it reads as "architecture-ready."
- Audit log: append-only JSONL, one entry per agent action.
- Execution timeline UI: render this log live, per task, as a checklist. This single component does more demo work than almost anything else — makes "agentic," "routing," "RAG," and "auditability" all visible in one glance.

FR-10 — Frontend (Workbench UI)

Not a conventional admin dashboard — an AI workbench.

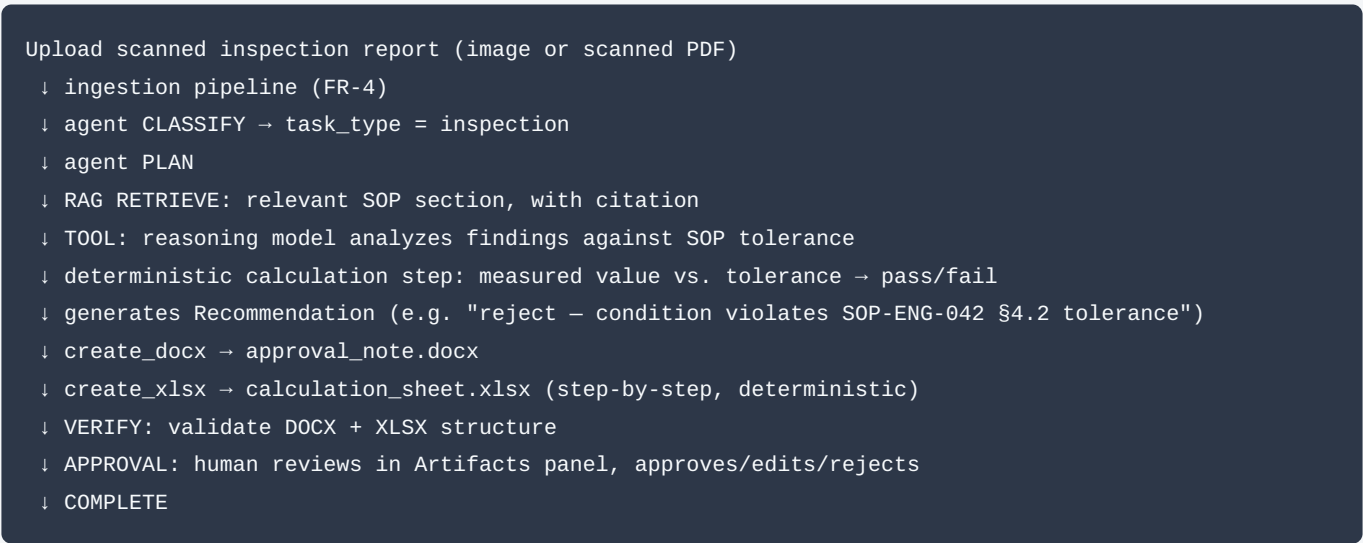


Build with React + Vite + Tailwind — deliberately skip Next.js/SSR complexity; one user, no SEO needs. A small routing-receipt badge ("Handled by: coding · qwen2.5-coder:7b") on every response is worth more demo credit than visual polish. The Context Panel adds a **P&ID Graph** tab that renders the overlay image and lets the graph JSON be inspected node-by-node.

7. Demo Workflows

Three flagship end-to-end workflows plus one differentiator, matching the PS's own four expected-solution bullets (agentic task / coding task / multimodal task / sovereignty proof) — with Workflow 4 now built as a *real* structured-extraction capability rather than a caption generator.

Workflow 1 — Inspection Report → Approval Note (+ Calculation Sheet)



Test fixture (ML owner, build first): one synthetic scanned inspection report with a deliberate OCR challenge (slight skew/noise), one measured value, one finding, and one matching SOP document in the knowledge base. Build the corpus before the pipeline — you cannot debug RAG citation quality against an empty knowledge base.

Workflow 2 — Coding → Sandboxed Verified Output

```
"Add pagination to this endpoint"
↓ agent CLASSIFY → task_type = coding
↓ ROUTE → coding model
↓ TOOL: read_file (existing code)
↓ TOOL: create_code (generate change)
↓ TOOL: execute_code + run_tests (sandbox, FR-6)
↓ VERIFY: tests fail (seed one deliberate failure for the demo)
↓ REPAIR → regenerate → re-run tests
↓ VERIFY: tests pass
↓ COMPLETE — return diff + passing test output
```

Test fixture: a tiny FastAPI/Flask endpoint file with an obvious missing-pagination bug and a test file that currently fails against it. Seed the failure deliberately so REPAIR is guaranteed to trigger on demo day.

Workflow 3 — Sovereignty Proof

```
Attempt outbound request from inside the sandbox (curl/wget to a public IP)
↓ BLOCKED at container network level
↓ event logged (telemetry) and shown on Sovereignty Monitor
↓ normal Workflow 1 or 2 continues in parallel
↓ zero external traffic recorded for the entire session
```

Cheapest of the four to build (mostly Docker flags + a small dashboard) and the highest leverage for a live audience — build it early enough to rehearse, not last.

Workflow 4 — P&ID Structured Understanding (NEW)

```
Upload a P&ID (public sample dataset image)
↓ agent CLASSIFY → task_type = pid_analysis
↓ ROUTE → vision model + pid pipeline
↓ TOOL: extract_pid_graph (FR-4B: detect → trace → assemble graph)
↓ VLM cross-check + grounded narrative ("explain the flow path")
↓ VERIFY: confidence gate; low-confidence regions flagged, not guessed
↓ output: graph JSON + annotated overlay + narrative shown in Context Panel
↓ COMPLETE (no approval gate needed — this is understanding, not a decision)
```

Test fixture: 2–3 curated public P&ID samples (per PS's own dataset guidance — "publicly available document samples... sample P&IDs from open datasets"), each hand-annotated with a small ground-truth node/edge list so accuracy can be checked and demoed honestly. This workflow can run standalone or be shown feeding into Workflow 1 if an inspection report references a P&ID excerpt.

8. API Contract (initial surface)

Endpoint	Method	Purpose
<code>/api/chat</code>	POST	Submit a user request, returns <code>task_id</code> , streams state-machine progress via SSE
<code>/api/files/upload</code>	POST	Upload a document/image, returns <code>file_id</code>
<code>/api/tasks/{task_id}</code>	GET	Full task state object (§5.2)
<code>/api/tasks/{task_id}/ timeline</code>	GET	Ordered audit-log entries for this task
<code>/api/tasks/{task_id}/ approve</code>	POST	Human approval/rejection/edit for an APPROVAL-state task
<code>/api/artifacts/ {artifact_id}</code>	GET	Download a generated file
<code>/api/knowledge/upload</code>	POST	Ingest a document into the RAG corpus
<code>/api/models</code>	GET	Current registry + which model is currently resident
<code>/api/monitoring/ sovereignty</code>	GET	Live sovereignty counters (FR-8)
<code>/api/pid/analyze</code>	POST	NEW — submit a P&ID image, returns graph JSON + overlay URL + narrative

Keep this list frozen once agreed — it's the contract between Backend A/B and the frontend; changing it mid-build is the single most common source of last-minute breakage on a small team.

9. Data & Storage

- Task state / audit log / users: SQLite — enough for a single-node offline prototype, don't stand up Postgres.
- Vector store: Qdrant, single Docker container, one collection per document corpus (e.g. `sops`, `manuals`).
- File storage: flat filesystem under a workspace directory, one subfolder per `task_id`, so the sandbox and artifact generators only touch a jailed path.
- P&ID graphs: stored as JSON alongside the task record (small enough not to need a graph database — no custom DB per Non-Goals); overlay images stored as files in the task's artifact folder next to the DOCX/XLSX outputs.
- Model weights: managed entirely by Ollama's own storage; don't build custom model file management.

10. Non-Functional Requirements

Requirement	How it's met
Fully local / no external calls	FR-8; verified by telemetry, not by absence of complaints
Multi-model, auto-selected	FR-1, visible routing receipt
Agentic execution with verification/correction	FR-2, bounded REPAIR loop
Multimodal local processing	FR-4, FR-4B
Local knowledge with citations	FR-5
Real deliverables (incl. calculation steps)	FR-7
Sandboxed execution	FR-6
Auditability	FR-9, execution timeline
Structured engineering-drawing understanding	FR-4B, Workflow 4
RBAC/SSO/multi-user	Explicitly out of scope — architecture leaves a <code>role</code> field, nothing more

11. Security Threat Model

Threat	Mitigation
Prompt injection via uploaded document	Retrieved/extracted content always treated as data, never system/instruction role; tool calls allow-listed per task type
Malicious PDF (parser exploit)	Parse in an isolated process; enforce page-count and file-size limits before parsing
Knowledge-base poisoning	Only ingest through <code>/api/knowledge/upload</code> ; tag source/trust metadata on every chunk; retrieval scoped to active corpus
Tool abuse / path traversal	All filesystem tools jailed to the per-task workspace directory; typed arguments only, no raw shell strings
Sandbox escape	Network-disabled, capability-dropped, resource-limited, ephemeral container (FR-6)
Infinite agent loop	Max-step counter + per-state timeout + explicit terminate state
	New: confidence gate on every node/edge; low-confidence elements shown greyed-out rather than guessed; VLM narrative is grounded on the extracted graph (not free-floating),

Threat	Mitigation
P&ID mis-extraction (wrong connection asserted)	so a wrong edge is traceable to a specific low-confidence detection instead of an unexplainable hallucination; graph output is presented as an aid for a human engineer, never as an auto-approved decision
Supply-chain / telemetry leakage from dependencies	Pin dependency versions; one-time check of installed packages for phone-home behavior before freezing the build

12. Pre-Build Checklist

Do these before writing application code — they de-risk the largest unknowns first:

- [] Confirm actual laptop GPU model + VRAM (run `nvidia-smi` or equivalent — don't guess)
- [] Pull and smoke-test all four LLMs in the roster (§4.2) on that exact machine
- [] Confirm Ollama model-swap works reliably and measure swap time (affects UX — frontend should show a "loading model..." state, not hang)
- [] Build the synthetic inspection-report + SOP corpus test fixture (Workflow 1)
- [] Build the seeded-failing-test coding fixture (Workflow 2)
- [] Verify Docker network isolation actually blocks egress (`docker run --network=none ... curl ...` should fail)
- [] Decide and confirm the demo machine's OS ahead of the presentation — determines host-level nftables vs. Docker-network-only proof for Workflow 3
- [] **New:** download 2–3 public P&ID sample images and hand-annotate a small node/edge ground truth for each (Workflow 4)
- [] **New:** decide symbol-detection approach (template matching vs. tiny YOLO) before building the tracer — don't build the graph assembly step against an undecided detector output format
- [] **New:** run the line-tracer on one clean sample end-to-end before touching the frontend graph view — this is the highest-technical-risk component in the whole build
- [] Freeze the API contract (§8) between all three team members

13. Explicit Non-Goals

Do not build any of the following — each is a credible-sounding scope trap that the PS does not require:

- Cloud dependency of any kind, even "optional" cloud fallback
- A giant/frontier local model "for quality" — completeness beats model size for this grading rubric
- Multi-agent swarms / unrestricted ReAct loops
- A custom inference engine or a custom vector database (P&ID graphs are stored as plain JSON, not a graph DB)

- Fine-tuning any model
 - Production-grade SSO/RBAC or Kubernetes/HA deployment — **confirmed:** PS explicitly says "single workstation or server," so multi-node/HA is out regardless of anything else
 - Browser automation
 - A sovereignty dashboard that isn't backed by real egress blocking — worse than not building one at all if a judge tests it
 - **A general-purpose P&ID parser that reliably handles arbitrary, unseen, or hand-annotated field scans.** What's in scope (FR-4B) is a structured parser tuned against a small set of curated public P&ID fixtures — genuinely real, genuinely structured, but not claimed to generalize beyond the demo corpus. Don't overclaim this on stage.
-

14. Build Milestones (dependency order — not time-boxed)

Each milestone's Definition of Done is "the described flow runs end-to-end on a live machine," not "the code exists." Don't start the next milestone until the current one is demoable.

1. **Skeleton:** Docker Compose brings up FastAPI (empty), Qdrant, Ollama with all 4 models pulled; frontend shell loads and hits a health endpoint. Sovereignty container network rule (`--network=none` on a throwaway container) proven to block egress.
2. **Model registry + router live:** a hardcoded request returns a visible routing receipt in the API response.
3. **Agent state machine + minimal tools:** `read_file` / `write_file` / `list_files` wired into the state machine; a trivial task completes the full INTAKE→COMPLETE path.
4. **RAG pipeline:** synthetic SOP corpus ingested; a query returns cited chunks from Qdrant.
5. **Ingestion pipeline:** scanned inspection report fixture → OCR → normalized text, with VLM fallback path tested at least once.
6. **Workflow 1 end-to-end:** scan → OCR → RAG → recommendation → DOCX + calculation XLSX → validated → shown in Artifacts panel → human approval gate works.
7. **Sandbox + Workflow 2 end-to-end:** code fixture → generate → execute → seeded test failure → repair → pass.
8. **Sovereignty dashboard (Workflow 3):** live counters, live blocked-request demo works reliably and repeatably.
9. **P&ID pipeline (Workflow 4) — run as a parallel track from Milestone 5 onward, owned primarily by ML Engineer, does not block Milestones 6–8:** a. Symbol detection tuned against the 2–3 curated fixtures. b. Line tracing produces edges on the cleanest fixture. c. Graph JSON assembled + confidence scoring. d. Overlay rendering wired into the frontend Context Panel. e. VLM cross-check/narrative grounded on the graph, confidence-gate fallback tested.
10. **Execution timeline UI + audit log** wired to all four workflows.
11. **Integration pass:** run all four workflows back-to-back without restarting anything; fix whatever breaks when they share state, GPU memory, or the sandbox at the same time — this always surfaces problems the individual workflows didn't.
12. **Fallback-path check:** deliberately kill the GPU/force CPU inference and confirm the system degrades instead of crashing.
13. **Full rehearsal:** all four workflows, in the order they'll be presented, on the actual demo machine.

Sequencing note: Workflow 4 is the highest-technical-risk item in this plan. It is deliberately placed as a parallel track rather than gating the three PS-mandated core workflows — if it runs behind schedule, Workflows 1–3 must still be fully demoable on their own, since those alone already satisfy every PS "Expected Solution" bullet at the minimum bar. Workflow 4 is the differentiator on top, not a dependency underneath.

15. Verification Notes

Checked against current sources before finalizing this document (23 Aug 2026):

- PS 26117 is confirmed as MRPL's listed SIH 2026 Smart Automation problem statement; the "Expected Solution" paragraph was re-read verbatim while building §1's traceability matrix to make sure nothing was missed.
- Single-node/no-Kubernetes decision re-confirmed directly against the PS's own "demonstrable on a single workstation or server" wording — not just against the research synthesis's team-size reasoning. Both sources agree; no conflict here.
- **P&ID full connectivity parser reinstated into scope by explicit decision.** Both the v1 Implementation PRD and the 9-model research synthesis put this in "what not to build," but that recommendation was scoped to raw hackathon-speed feasibility, not to PS requirements — the PS itself never excludes it and explicitly names P&IDs as its headline confidential-data example. The honest caveat carried into FR-4B and §13 is that the *parser is tuned to curated public fixtures*, not claimed to generalize — this keeps the added scope real rather than a demo-day gamble.
- All other "what not to build" items (RBAC/SSO, multi-agent swarms, custom inference engine, custom vector DB, fine-tuning, browser automation, giant model, production HA) were re-checked against the PS text specifically and confirmed to have no PS-side requirement backing them — they stay excluded.
- Model recommendation stays laptop-appropriate (Section 4) rather than the research synthesis's generic "27–32B on a 24GB card" — team's actual hardware is a laptop GPU. Qwen3-8B / Qwen2.5-Coder-7B / Qwen2.5-VL-7B (and smaller siblings) remain current, actively maintained, Ollama-installable models as of Aug 2026.
- Qdrant remains a reasonable vector DB choice for an agentic RAG loop at this scale.
- PaddleOCR vs. Tesseract: Tesseract stays primary for install reliability on unknown laptop OS/driver combinations under team-of-3 constraints — not a quality claim. PaddleOCR remains a stretch upgrade for table/layout-heavy documents if time allows.
- Sandbox: hardened standard Docker stays over rootless Podman + seccomp for the same team-size/setup-time reason; the security guarantees required by the PS (no network, resource limits, isolation) are unaffected.

Re-verify before the demo: exact model VRAM figures and Ollama swap latency on the actual laptop; P&ID detector/tracer accuracy on the exact fixtures chosen; firewall rules on the exact demo machine OS. Treat none of this document's specifics as frozen hardware/software fact — re-check on the day.